

# Cheatsheet Definitivo — TD2

STM32F103C8T6 (BluePill) · ARM Cortex-M3 · 72 MHz

Resumen Conceptual (Sin Código) para el Parcial

## 1 Mapa de Memoria

| Periferico | Dir. Base   |
|------------|-------------|
| GPIOA      | 0x4001 0800 |
| GPIOB      | 0x4001 0C00 |
| GPIOC      | 0x4001 1000 |
| AFIO       | 0x4001 0000 |
| EXTI       | 0x4001 0400 |
| RCC        | 0x4002 1000 |
| ADC1       | 0x4001 2400 |
| USART1     | 0x4001 3800 |
| TIM2       | 0x4000 0000 |
| SysTick    | 0xE000 E010 |
| NVIC       | 0xE000 E100 |
| FLASH      | 0x4002 2000 |

**Zonas clave:** Flash: 0x0800\_0000, SRAM: 0x2000\_0000,  
Perifericos: 0x4000\_0000, Core: 0xE000\_0000.

## 2 RCC — Sistema de Relojos

### 2.1 Clock Tree (72 MHz)

HSI (8 MHz int.) o HSE (8 MHz cristal) → PLL (×9) → SYSCLK = 72 MHz  
AHB = SYSCLK/1 = 72 MHz; APB2 = 72 MHz ("rapido")  
APB1 = SYSCLK/2 = 36 MHz ("lento": TIM2, USART2/3)  
**OJO:** Si APB1 prescaler ≠ 1, el timer recibe ×2 → TIM2 = 72 MHz

Conceptual: Configurar PLL a 72 MHz

- Encender el cristal externo (HSE) en el registro RCC\_CR.
- Esperar a que la flag HSERDY indique que el reloj es estable.
- Configurar la latencia de la Flash (2 wait states) y encender el prefetch buffer en FLASH\_ACR.
- Ajustar prescalers de buses en RCC\_CFGR: AHB (DIV1), APB1 (DIV2), APB2 (DIV1).
- Configurar el PLL en RCC\_CFGR: fuente = HSE, multiplicador = ×9.
- Encender el PLL (PLLON) y esperar flag PLLRDY.
- Cambiar la fuente del SYSCLK para que sea el PLL (SW) y esperar a que el hardware lo confirme (SWS).

### 2.2 Habilitar Relojos de Perifericos

| Reg.    | Bit  | Periferico |
|---------|------|------------|
| APB2ENR | [0]  | AFIO       |
| APB2ENR | [2]  | GPIOA      |
| APB2ENR | [3]  | GPIOB      |
| APB2ENR | [4]  | GPIOC      |
| APB2ENR | [9]  | ADC1       |
| APB2ENR | [14] | USART1     |
| APB1ENR | [0]  | TIM2       |

**Paso fundamental:** Antes de configurar cualquier periférico (incluyendo puertos GPIO), se debe prender su bit correspondiente en el registro APB1ENR o APB2ENR del RCC.

## 3 GPIO

### 3.1 Modos (4 bits por pin en CRL/CRH)

CRL = pines 0–7, CRH = pines 8–15.  
Cada pin usa 4 bits: [CNF1 CNF0 MODE1 MODE0]

| CNF | MODE | Modo                     | Hex |
|-----|------|--------------------------|-----|
| 00  | 00   | Entrada analogica        | 0x0 |
| 01  | 00   | Entrada flotante (reset) | 0x4 |
| 10  | 00   | Entrada pull-up/down     | 0x8 |
| 00  | 01   | Salida PP 2 MHz          | 0x1 |
| 00  | 10   | Salida PP 2 MHz          | 0x2 |
| 00  | 11   | Salida PP 50 MHz         | 0x3 |
| 10  | 01   | AF Push-Pull 10 MHz      | 0x9 |
| 10  | 11   | AF Push-Pull 50 MHz      | 0xB |

**Pull-up/down:** con CNF=10, MODE=00 (0x8), se elige con ODR:  
ODR bit=1 → Pull-Up; ODR bit=0 → Pull-Down.

### 3.2 Registros de Datos

| Reg.        | Uso                                 |
|-------------|-------------------------------------|
| IDR         | Leer estado de pines (solo lectura) |
| ODR         | Escribir salidas / seleccionar pull |
| BSRR[15:0]  | Set atomico (escribir 1 → pin=1)    |
| BSRR[31:16] | Reset atomico (escribir 1 → pin=0)  |

Conceptual: Configurar un pin GPIO

- Identificar el puerto y encender su reloj en RCC→APB2ENR.
- Determinar si el pin es del 0 al 7 (usar registro CRL) o del 8 al 15 (usar registro CRH).
- Calcular la posicion de los 4 bits del pin. Para el pin  $p$  en CRL es  $4 \times p$ . En CRH es  $4 \times (p - 8)$ .
- Aplicar una mascara (0xF corrida a la posicion) en AND negado (&~ ") para limpiar la configuracion anterior del pin.
- Escribir el nuevo valor hexadecimal del modo deseado (ver tabla arriba) mediante un OR (|=), corrido a la misma posicion.
- Si el modo es Entrada Pull-Up / Pull-Down (0x8), se debe escribir un 1 o un 0 en el registro ODR para elegir cual de las dos resistencias habilitar.

!! Posicion de bits en CRL/CRH

Ej: PA9 → CRH, pos =  $4 \times (9 - 8) = 4$   
Ej: PA2 → CRL, pos =  $4 \times 2 = 8$

## 4 SysTick

### 4.1 Registros

| Reg. | Bits           | Funcion                        |
|------|----------------|--------------------------------|
| CTRL | [0] ENABLE     | Enciende el contador           |
| CTRL | [1] TICKINT    | Habilita interrupcion          |
| CTRL | [2] CLKSOURCE  | 1=AHB, 0=AHB/8                 |
| CTRL | [16] COUNTFLAG | 1=Llego a cero                 |
| LOAD | [23:0]         | Valor de recarga               |
| VAL  | [23:0]         | Valor actual (escribir limpia) |

Formula del LOAD

$$\text{LOAD} = (T_{\text{deseado}} \times f_{\text{clock}}) - 1$$

| Periodo | CLKSRC=1 (72M) | CLKSRC=0 (9M) |
|---------|----------------|---------------|
| 100 μs  | 7199           | 899           |
| 1 ms    | 71999          | 8999          |
| 10 ms   | 719999         | 89999         |

Conceptual: Configurar SysTick

- Seleccionar la fuente de reloj en el bit CLKSOURCE del SysTick→CTRL. Puede ser el core (AHB) o AHB/8.
- Calcular el valor de LOAD con la fórmula para obtener el periodo deseado. Escribir este valor en el registro LOAD.
- Reiniciar el contador actual escribiendo un 0 en el registro VAL (cualquier escritura limpia la bandera y el conteo).
- Prender la interrupción (TICKINT) y encender el timer (ENABLE) simultáneamente en el registro CTRL.
- Escribir el Handler de interrupción con el nombre exacto de la CMSIS (SysTick\_Handler).

## 5 EXTI — Interrupciones Externas

### 5.1 Mapeo de lineas EXTI

| Linea      | IRQn | Handler              |
|------------|------|----------------------|
| EXTI0      | 6    | EXTI0_IRQHandler     |
| EXTI1      | 7    | EXTI1_IRQHandler     |
| EXTI2      | 8    | EXTI2_IRQHandler     |
| EXTI3      | 9    | EXTI3_IRQHandler     |
| EXTI4      | 10   | EXTI4_IRQHandler     |
| EXTI5..9   | 23   | EXTI9_5_IRQHandler   |
| EXTI10..15 | 40   | EXTI15_10_IRQHandler |

**AFIO\_EXTICR:** EXTICR[0] = pines 0–3, EXTICR[1] = 4–7, etc.

4 bits por pin: 0000=PA, 0001=PB, 0010=PC

Conceptual: Configurar EXTI (Ej: en un boton)

- Habilitar el reloj del puerto GPIO y fundamentalmente el del bloque AFIO.
- Configurar el pin físico del botón como entrada (flotante, pull-up o pull-down según el circuito eléctrico).
- Mapear el pin a la línea EXTI correspondiente usando los registros AFIO→EXTICR. Ej: PA0 a EXTI0.
- Desenmascarar (habilitar) la interrupción en el registro EXTI→IMR.
- Elegir el evento de disparo habilitando la detección de flanco ascendente (RTSR) y/o descendente (FTSR).
- Configurar la prioridad deseada en el controlador NVIC y habilitar globalmente la línea (Ej: EXTI0\_IRQn).
- En el handler (EXTI0\_IRQHandler), lo primero que se debe hacer es **limpiar la bandera de pendiente** escribiendo un '1' en el registro EXTI→PR para evitar que la interrupción se dispare en bucle infinito.

## 6 NVIC

| Funcion CMSIS         | Que hace              |
|-----------------------|-----------------------|
| NVIC_EnableIRQ(n)     | Habilita IRQ $n$      |
| NVIC_DisableIRQ(n)    | Deshabilita IRQ $n$   |
| NVIC_SetPriority(n,p) | Prioridad $p$ (0=max) |
| NVIC_SetPendingIRQ(n) | Fuerza IRQ pendiente  |

**IRQn de referencia rapida:**  
EXTI0=6, EXTI1=7, ADC1\_2=18, TIM2=28,  
USART1=37, EXTI15\_10=40.

**Tail-chaining:** Si hay otra IRQ pendiente al salir de una ISR, el core NO restaura/guarda registros, salta directo (ahorra 12 ciclos).  
**Late arrival:** Si llega una IRQ de mayor prioridad durante el stacking, se atiende primero sin deshacer el stacking ya comenzado.

## 7 ADC1 — Conversor Analogico

### 7.1 Registros clave

| Registro | Bits importantes                 |
|----------|----------------------------------|
| CR2      | [0] ADON, [1] CONT, [22] SWSTART |
| CR2      | [23] TSVREFE, [20] EXTTRIG       |
| CR2      | [19:17] EXTSEL (111=SWSTART)     |
| SR       | [1] EOC (dato listo)             |
| SQR3     | [4:0] SQ1 (nro de canal)         |
| DR       | [15:0] Dato convertido           |
| SMPR1/2  | Tiempo de muestreo por canal     |

Formulas ADC

$$V_{\text{medido}} = \frac{\text{ADC\_DR} \times 3,3\text{V}}{4095}$$

ADC clock ≤ 14 MHz. Con PCLK2=72 MHz:  
Prescaler /6 → 12 MHz (OK)

Conceptual: Inicializar y leer el ADC1

- Habilitar clocks del puerto analógico (GPIOA) y del ADC1.
- Configurar el pin físico (Ej: PA4) en modo Entrada Analógica (valor 0x0 en CRL/CRH).
- Encender el bloque ADC seteando el bit ADON en el CR2. (Si se lee temperatura interna, prender también TSVREFE).
- Configurar el tiempo de muestreo del canal específico en SMPR1 o SMPR2 (Ej: usar el muestreo más largo para mayor precisión).
- Configurar el origen de disparo (EXTSEL). Para disparo por software, configurar como 111 y prender EXTTRIG.
- Para leer un dato (Loop de programa):**
  - Seleccionar el canal a leer escribiendo su número en el campo SQ1 del registro de secuencia SQR3.
  - Disparar la conversión encendiendo el bit SWSTART en el CR2.
  - Hacer un polling bloqueante (bucle while) esperando que el bit EOC (End of Conversion) del registro de estado SR se ponga en 1.
  - Leer y guardar el valor resultante desde el registro DR.

**Canales utiles:** CH0=PA0, CH1=PA1, CH4=PA4,  
CH16=Sensor Temp interno, CH17=V\_REFINT.

## 8 USART1 — Comunicacion Serie

### 8.1 Registros

| Registro | Bits clave                           |
|----------|--------------------------------------|
| CR1      | [13] UE, [3] TE, [2] RE              |
| CR1      | [5] RXNEIE (int. recepcion)          |
| CR1      | [12] M (0=8bits), [10] PCE (paridad) |
| CR2      | [13:12] STOP (00=1 bit)              |
| BRR      | [15:4] Mantissa, [3:0] Fraccion      |
| SR       | [7] TXE, [6] TC, [5] RXNE            |
| DR       | [8:0] Dato TX/RX                     |

Formula Baud Rate (BRR)

$$\text{USARTDIV} = \frac{f_{\text{PCLK2}}}{16 \times \text{BaudRate}}$$

| Baud   | USARTDIV | BRR (hex) |
|--------|----------|-----------|
| 9600   | 468.75   | 0x1D4C    |
| 19200  | 234.375  | 0x0EA6    |
| 115200 | 39.0625  | 0x0271    |

Mantissa = parte entera, Fraccion = parte decimal  $\times$  16

Conceptual: Inicializar USART1

- Habilitar reloj del USART1, puerto de pines (GPIOA) y AFIO.
- Configurar el pin **TX** (Ej. PA9) como Función Alternativa Push-Pull (valor 0xB a 50MHz).
- Configurar el pin **RX** (Ej. PA10) como Entrada Flotante (valor 0x4).
- Configurar la estructura de la trama en el CR1 y CR2: (Largo de palabra M, control de paridad PCE, cantidad de bits de stop STOP).
- Calcular el divisor de baudios e inyectarlo en el registro BRR (dividiendo el valor flotante en parte entera para la mantisa y decimal para la fracción).
- Habilitar globalmente el módulo USART (UE), el transmisor (TE) y el receptor (RE) en el CR1.
- Si se usa interrupción por recepción (común), habilitar bit RXNEIE y encender la IRQ en el NVIC.
- Para transmitir (Polling):** Esperar a que la flag TXE en el SR indique buffer vacío, y luego escribir el byte en el registro DR.
- Para recibir (ISR):** En el handler, verificar la flag RXNE y guardar en una variable lo leído del registro DR.

9 TIM2 — PWM por Hardware

9.1 Registros

| Registro | Bits clave                              |
|----------|-----------------------------------------|
| CR1      | [0] CEN (habilita timer)                |
| PSC      | [15:0] Prescaler (divisor - 1)          |
| ARR      | [15:0] Auto-reload (periodo - 1)        |
| CCRx     | [15:0] Comparacion (duty)               |
| CCMRz    | OCxM (modo), OCxPE (preload)            |
| CCER     | CCxE (habilita canal), CCxP (polaridad) |
| EGR      | [0] UG (forzar update)                  |

z=1 para canales 1,2 · z=2 para canales 3,4

x = numero de canal (1 a 4)

Formulas PWM

$$f_{\text{PWM}} = \frac{f_{\text{TIM}}}{(\text{PSC} + 1) \times (\text{ARR} + 1)}$$
$$\text{Duty \%} = \frac{\text{CCR}_x}{\text{ARR} + 1} \times 100 \%$$

Ej: 1 kHz con  $f_{\text{TIM}}=72\text{ MHz}$ :  
 $\text{PSC}=71, \text{ARR}=999 \rightarrow \frac{72\text{M}}{72 \times 1000} = 1\text{ kHz}$   
50 % duty  $\rightarrow \text{CCR}_x = 500$

Conceptual: PWM en TIM2 (Canal 3)

- Habilitar reloj del timer (APB1ENR) y puerto destino.
- Configurar el pin físico (Ej: PA2 para CH3) como Función Alternativa Push-Pull (0xB).
- Setear la frecuencia estableciendo el pre-escalador (PSC) y el valor máximo de recarga (ARR).
- Definir el ancho de pulso inicial asignando un valor al registro de comparación del canal correspondiente (Ej: CCR3).
- Configurar el modo del canal como **PWM Mode 1** en el registro CCMR correspondiente (Ej. CCMR2 para el CH3) mediante los bits OCxM. (También activar el PreLoad en el mismo registro).
- Habilitar la salida física del canal encendiendo el bit CCxE en el registro CCER.
- Forzar una actualización de los shadow registers escribiendo un 1 en el bit UG del registro EGR.
- Arrancar a contar encendiendo el timer con el bit CEN en el CR1.

**Pines PWM de TIM2:**  
CH1=PA0, CH2=PA1, CH3=PA2, CH4=PA3  
**PWM Mode 1** (OC1M=110): Activo mientras CNT < CCRx

10 Pines Alternativos (Referencia)

| Pin  | Funcion Alternativa       | Config    |
|------|---------------------------|-----------|
| PA0  | TIM2_CH1 / EXTI0          | 0xB / 0x8 |
| PA1  | TIM2_CH2 / EXTI1          | 0xB / 0x8 |
| PA2  | TIM2_CH3 / USART2_TX      | 0xB       |
| PA3  | TIM2_CH4 / USART2_RX      | 0x4       |
| PA4  | ADC CH4 / SPI1_NSS        | 0x0       |
| PA9  | USART1_TX                 | 0xB       |
| PA10 | USART1_RX                 | 0x4       |
| PC13 | LED onboard (activo bajo) | 0x2       |

11 Operaciones Logicas (Conceptuales)

- Al operar con registros (|=, &=, ^=, ~=), se aplica:
- Prender un bit específico:** Se usa la máscara base (el 1 desplazado) aplicando una operación **OR**. El resto de bits queda inalterado por ser operados con 0.
  - Apagar un bit específico:** Se usa la máscara base, se la invierte lógicamente (**NOT**, pasando todos a 1 salvo el bit a apagar que será 0) y se le aplica una operación **AND**.
  - Leer o comprobar un bit:** Se usa la máscara base y se le aplica un **AND** contra el registro para enmascarar (eliminar) el resto de bits y comprobar si el resultado es distinto de 0.
  - Modificar un campo de múltiples bits (ej: 4 bits en GPIO):** Siempre se debe primero **limpiar** el campo completo usando AND con una máscara invertida de longitud correspondiente (0xF) y luego **escribir** la nueva configuración usando un OR.

Basado en RM0008, PM0056 y el TP2 de la catedra.

Imprimible en A4 vertical, 2 columnas.